A Quoted Alphabet: Stage One of a Verified Reader

Working paper v0.1, Project secret3, August 11, 2026

Abstract

Line C5 of the research agenda asks for a verified self-interpreter: a term U over S and K that, handed the quotation of any closed program, simulates it, with the whole loop machine-checked. This paper delivers the campaign's first stage, the quotation layer. Closed programs become data through a three-constructor code (Code, with embed giving each code its denoted term), and a Scott-style quotation maps every code to a closed selector term. The layer's application programming interface is three compiled behavior theorems: the quote of S selects its first branch in six moves (qSCase), the quote of K its second in two (qKCase), and the quote of an application selects the third branch and hands it both quoted children in eight (qAppCase), the last by literal reuse of the workshop pair from the project's book, whose opening theorem pairBeta becomes the quotation's spine. Two data-discipline theorems complete the stage: quotes are closed, no variable occurs in any quote (quoteClosed), and quotation is injective, distinct codes have distinct quotes (quoteInj). Stage two, the step simulator, is specified against this interface with its risks stated.

1. The campaign

A universal term is the cash value of the 1936 completeness results: the machine that runs descriptions, so that the word suffices for the act. The project's book leans on the classical fact; line C5 asks for the project's own, compiled. The campaign order is dictated by dependency: before anything can read, there must be something to read, a representation of programs as data with enough verified structure that a reader's correctness proof can stand on it. This paper is that representation. The design constraint throughout is stage two's induction: every choice below is made so that a future simulator can case on a quote, extract children, and recurse, with each capability already a named theorem rather than a hope.

2. The design

The alphabet of closed programs is the obvious one:

```
data Code = QS | QK | QApp Code Code

embed : Code -> Term
embed QS = CS
embed QK = CK
embed (QApp x y) = App (embed x) (embed y)
```

Quotation is Scott-style: a datum is the term that, offered one continuation per constructor, applies the right one to its components. This is the encoding under which case analysis is cheapest, which is exactly what a simulator does at every step. The three quotes:

```
quote QS         = B K K
quote QK         = K K
quote (QApp x y) = K (K (pair (quote x) (quote y)))
```

written above in combinator shorthand; in the module, B and pair are the long-standing citizens bC and pairC. Two of the three lines deserve a sentence each. The quote of K is two symbols, the term that discards one argument and keeps the next, which is the second-branch selector by nature. And the quote of an application is the project's own box: the workshop chapter of the book built pair x y as seven symbols with a six-move opening theorem, and the quotation layer now wraps that box in two K's so that the first two branches are discarded and the third receives the box's contents. The book's pedagogy has become the program's infrastructure, and the reuse is not cosmetic: qAppCase's proof is two drops followed by pairBeta, unchanged.

3. The interface

```
qSCase   : Steps (quote QS  s k a) s                              -- 6 moves
qKCase   : Steps (quote QK  s k a) k                              -- 2 moves
qAppCase : Steps (quote (QApp x y) s k a) (a (quote x) (quote y)) -- 8 moves
```

(display curried for readability; the module states them with explicit App spines). These three theorems are the entire interaction a reader needs with a quote: to inspect a program is to apply its quotation to three continuations, and the right one fires, with the children delivered in the application case. All three are explicit move chains in the module's house style, positions and all; no simulation, no admissibility, no meta-theory, just the moves.

4. Data discipline

Two theorems make the alphabet safe to compute with. quoteClosed: for every variable name and every code, the occurrence count of that name in the quote is zero, by the occurrence ledger of the linearity paper; quotes are pure closed data, so the abstraction engine of the vanishing-engine paper can build readers that mention quotes without capture anxiety. quoteInj: quotation is injective, proved by constructor-injectivity descent through the quote shapes; distinct programs never share a name, so a future correctness statement about U applied to a quote determines the program it speaks of.

5. Stage two, specified honestly

The next stage is a one-step simulator: a term Sim with the theorem that for every code c whose embedded term takes a step to u, Sim applied to quote c walks to the quote of a code embedding u. The design against this paper's interface: Sim cases on its argument via the three continuations; the S and K branches return tagged normal indicators; the application branch receives both children and must itself case on the left child's shape to detect the two redexes, which is where the real work lives, a nested three-way case over quoted structure, buildable with the vanishing engine and priced by its theorems. The stated risks: redex detection over Scott codes needs equality-of-shape tests that must be built from selectors, and the correctness induction will be long. The stated fallback, unchanged from the agenda: a meta-level interpreter with quotation theorems if the internal Sim resists. What this stage guarantees regardless: the alphabet, its API, and its discipline theorems stand on their own and serve any reader design.

Appendix A: Methods

All theorems are in the shared proof module Proofs.idr, 1,886 lines after this paper's additions, %default total, no postulates, no holes; idris2 —check Proofs.idr is the entire verification, and the entry point prints the theorem inventory, including qSCase, qKCase, qAppCase, quoteInj, and quoteClosed.

Appendix B: The quotation section (module part Q)

```
--------------------------------------------------------------------
-- Tier 2, part Q: the quoted alphabet (C5 stage one)
--------------------------------------------------------------------

||| closed programs as data: the alphabet the reader will read
data Code = QS | QK | QApp Code Code

||| what a code denotes: the closed term it names
embed : Code -> Term
embed QS         = CS
embed QK         = CK
embed (QApp x y) = App (embed x) (embed y)

||| Scott quotation: each code becomes a three-way selector.
||| The quote of an application is the workshop's box, wrapped
||| to ignore the first two branches: the book's pair is now
||| the quotation's spine.
quote : Code -> Term
quote QS         = App (App Proofs.bC CK) CK
quote QK         = App CK CK
quote (QApp x y) = App CK (App CK (Proofs.pairC (quote x) (quote y)))

||| branch behavior: the quote of S selects the first branch
qSCase : (s, k, a : Term) ->
  Steps (App (App (App (Proofs.quote QS) s) k) a) s
qSCase s k a =
  Then (CtxL (CtxL (CtxL (CtxL RedS)))) $
  Then (CtxL (CtxL (CtxL (CtxL (CtxL RedK))))) $
  Then (CtxL (CtxL RedS)) $
  Then (CtxL (CtxL (CtxL RedK))) $
  Then (CtxL RedK) $
  Then RedK Done

||| the quote of K selects the second branch
qKCase : (s, k, a : Term) ->
  Steps (App (App (App (Proofs.quote QK) s) k) a) k
qKCase s k a =
  Then (CtxL (CtxL RedK)) $
  Then RedK Done

||| the quote of an application selects the third branch and hands
||| it both children: two drops, then the box opens
qAppCase : (x, y : Code) -> (s, k, a : Term) ->
  Steps (App (App (App (Proofs.quote (QApp x y)) s) k) a)
        (App (App a (Proofs.quote x)) (Proofs.quote y))
qAppCase x y s k a =
  Then (CtxL (CtxL RedK)) $
  Then (CtxL RedK) $
  pairBeta (quote x) (quote y) a

||| quotes are closed data: no variable occurs in any quote
quoteClosed : (v : String) -> (c : Code) -> occ v (Proofs.quote c) = 0
quoteClosed v QS         = Refl
quoteClosed v QK         = Refl
```

```
quoteClosed v (QApp x y) =
  rewrite quoteClosed v x in rewrite quoteClosed v y in Refl

appInj1 : App a b = App c d -> a = c
appInj1 Refl = Refl

appInj2 : App a b = App c d -> b = d
appInj2 Refl = Refl

voidAppK : App a b = CK -> Void
voidAppK Refl impossible

voidAppS : App a b = CS -> Void
voidAppS Refl impossible

||| quotation is injective: distinct codes have distinct quotes
quoteInj : (a, b : Code) -> Proofs.quote a = Proofs.quote b -> a = b
quoteInj QS QS _ = Refl
quoteInj QK QK _ = Refl
quoteInj (QApp x y) (QApp x' y') prf =
  let inner  = appInj2 prf
      inner2 = appInj2 inner
      lhalf  = appInj1 inner2
      rhalf  = appInj2 inner2
      ex = quoteInj x x' (appInj2 (appInj2 (appInj2 lhalf)))
      ey = quoteInj y y' (appInj2 rhalf)
  in rewrite ex in rewrite ey in Refl
quoteInj QS QK prf = absurd (voidAppK (appInj1 prf))
quoteInj QK QS prf = absurd (voidAppK (sym (appInj1 prf)))
quoteInj QS (QApp x y) prf = absurd (voidAppK (sym (appInj2 prf)))
quoteInj (QApp x y) QS prf = absurd (voidAppK (appInj2 prf))
quoteInj QK (QApp x y) prf = absurd (voidAppK (sym (appInj2 prf)))
quoteInj (QApp x y) QK prf = absurd (voidAppK (appInj2 prf))
```

References

1. D. Scott, unpublished lectures; the encoding is folklore under his name, see M. Abadi, L. Cardelli, and G. Plotkin, ‘Types for the Scott numerals’, note, 1993.
2. T. Mogensen, ‘Efficient self-interpretation in lambda calculus’, Journal of Functional Programming 2 (1992), 345-364.
3. S. C. Kleene, ‘Lambda-definability and recursiveness’, Duke Mathematical Journal 2 (1936), 340-353.
4. Project secret3, ‘The Vanishing Engine, Verified’, in the module, part N, with chapter two of Rise of Form, August 2026.
5. Project secret3, research agenda v0.7, line C5, August 11, 2026.